```
  var xhr = new XMLHttpRequest();
  if (!('withCredentials' in xhr)) {
    alert('Browser does not support CORS.');
    return;
  }
  xhr.open(method, url);

  xhr.onerror = function() {
    alert('There was an error.');
  };

  xhr.onload = function() {
    var data = JSON.parse(xhr.responseText);
    if (data.stat == 'ok') {
      var photosDiv = document.getElementById('photos');
      photosDiv.innerHTML = '';
      var photos = data.photos.photo;
      for (var i = 0; i < photos.length; i++) {
        var img = document.createElement('img');
        img.src = photos[i].url_q;
        photosDiv.appendChild(img);
      }
    } else {
      alert(data.message);
    }
  };

  xhr.send();
}
</script>
</body>
</html>
```

Assigns a method to onload event handler

When processing the HTTP response, the code has access to various response variables, such as the HTTP status code, the HTTP status text, the response body, and the response HTTP headers. For example, the code in listing 2.4 uses the `responseText` property to grab the body of the response. Table 2.3 gives an overview of the properties that are available on the response.

Table 2.3 Response properties on the `XMLHttpRequest` object

Response property	Description
`status`	The HTTP status code (for example, 200 for a successful request).
`statusText`	The response string returned by the server (for example, `OK` for a successful request).
`response`	The body of the response, in the format defined by `responseType`. If the client indicated that the response type is `json`, the response will be a JSON object parsed from the response body.
`responseText`	The body of the response as a string. Can only be used if `responseType` was not set or was set as `text`.
`responseXML`	The body of the response as a DOM element (XML is here for historical reasons). Can only be used if `responseType` was not set or was set as `document`.